

TP multithreads

Lors de ce TP, nous avons mis en place différentes architectures de multithreads pour exécuter deux tâches sur une liste : trouver le maximum de la liste et compter le nombre de valeur entre VMIN et VMAX.

Première méthode :

La première solution a été de diviser la liste en sous-liste de taille "tasksize" ("tasksize" étant la taille maximale de la liste que pouvait accepter un thread). Nous avons ensuite créé autant de threads que de sous-listes et assigné chaque thread à une sous-liste.

Enfin nous avons récupéré les résultats des threads pour obtenir le résultat sur toute la liste.

Nous avons pu constater que cette technique est plus lente que la version séquentielle. Cela est dû au fait que tous les threads n'ont pas la même taille, le dernier est possiblement plus court que les autres. Cela oblige donc le dernier thread à attendre que les autres threads finissent. Cette perte de temps rend cette technique moins efficace que la méthode séquentielle. De plus cette méthode requiert un grand nombre de threads différents ($\text{liste.length} / \text{tasksize}$), ce qui peut être un problème pour le processeur.

deuxième méthode :

La deuxième solution reprend la première solution mais crée un ensemble de thread de taille "poolsize". Chaque thread se voit attribué une tâche. S'il y a plus de tâches que de threads, un thread se voit réattribué une tâche une fois la sienne finie.

De la même manière, nous récupérerons les résultats renvoyés par les threads afin de trouver le résultat sur toute la liste.

Cette technique ne résout pas le problème d'efficacité temporelle de la première solution mais permet de contrôler le nombre de threads que la méthode crée. Cette solution évite donc de surcharger le processeur.

deuxième méthode :

La dernière solution est de fonctionner de façon récursive par division successive de la liste. Si la liste à traiter est de taille inférieure à "tasksize", la méthode traite l'entièreté de la liste. Sinon, la méthode divise la liste en deux, crée deux threads qui appellent la méthode sur les deux moitiés de la liste. La méthode principale récupère les deux résultats des deux nouveaux threads pour trouver le résultat sur la liste complète.

Cette méthode résout le problème d'efficacité des deux premières méthodes car les threads ont tous la même taille. Cependant cette méthode crée énormément de différents threads, ce qui peut être un problème pour le processeur.